

Lecture 15

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

August 20, 2026

LALR Parse Table

- Look Ahead LR parsers

LALR Parse Table

- Look Ahead LR parsers
- Consider a pair of similar looking states (same kernel and different lookaheads) in the set of LR(1) items

LALR Parse Table

- Look Ahead LR parsers
- Consider a pair of similar looking states (same kernel and different lookaheads) in the set of LR(1) items
  $I_4 : C \rightarrow d., \quad c/d$
  $I_7 : C \rightarrow d., \quad \$$

LALR Parse Table

- Look Ahead LR parsers
- Consider a pair of similar looking states (same kernel and different lookaheads) in the set of LR(1) items
  $I_4 : C \rightarrow d.,\quad c/d$
  $I_7 : C \rightarrow d.,\quad \$$
- Replace I_4 and I_7 by a new state I_{47} consisting of $(C \rightarrow d.,\quad c/d/\$)$

LALR Parse Table

- Look Ahead LR parsers
- Consider a pair of similar looking states (same kernel and different lookaheads) in the set of LR(1) items
  $I_4 : C \rightarrow d.,\quad c/d$
  $I_7 : C \rightarrow d.,\quad \$$
- Replace I_4 and I_7 by a new state I_{47} consisting of $(C \rightarrow d.,\quad c/d/\$)$
- Similarly I_3 & I_6 and I_8 & I_9 form pairs

LALR Parse Table

- Look Ahead LR parsers
- Consider a pair of similar looking states (same kernel and different lookaheads) in the set of LR(1) items
  $I_4 : C \rightarrow d.,\quad c/d$
  $I_7 : C \rightarrow d.,\quad \$$
- Replace I_4 and I_7 by a new state I_{47} consisting of $(C \rightarrow d.,\quad c/d/\$)$
- Similarly I_3 & I_6 and I_8 & I_9 form pairs
- Merge LR(1) items having the same core

Construct LALR parse table

- Construct $C = I_0, \cdots, I_n$ set of LR(1) items

Construct LALR parse table

- Construct $C = I_0, \cdots, I_n$ set of LR(1) items
- For each core present in LR(1) items find all sets having the same core and replace these sets by their union

Construct LALR parse table

- Construct $C = I_0, \cdots, I_n$ set of LR(1) items
- For each core present in LR(1) items find all sets having the same core and replace these sets by their union
- Let $C' = J_0, \cdots, J_m$ be the resulting set of items

Construct LALR parse table

- Construct $C = I_0, \cdots, I_n$ set of LR(1) items
- For each core present in LR(1) items find all sets having the same core and replace these sets by their union
- Let $C' = J_0, \cdots, J_m$ be the resulting set of items
- Construct action table as was done earlier

Construct LALR parse table

- Construct $C = I_0, \cdots, I_n$ set of LR(1) items
- For each core present in LR(1) items find all sets having the same core and replace these sets by their union
- Let $C' = J_0, \cdots, J_m$ be the resulting set of items
- Construct action table as was done earlier
- Let $J = I_1 \cup I_2 \cdots \cup I_k$
  since $I_1, I_2 \cdots, I_k$ have same core, $goto(J, X)$ will have the same core
  Let $K = goto(I_1, X) \cup goto(I_2, X) \cdots \cup goto(I_k, X)$ the $goto(J, X) = K$

LALR parse table ...

State	c	d	$	S	C
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

Important points

- SLR and LALR parse tables have same number of states.

Important points

- SLR and LALR parse tables have same number of states.
- Merging items never produces shift/reduce conflicts but may produce reduce/reduce conflicts.

Important points

- SLR and LALR parse tables have same number of states.
- Merging items never produces shift/reduce conflicts but may produce reduce/reduce conflicts.
- New conflicts can not be of shift reduce kind:

Important points

- SLR and LALR parse tables have same number of states.
- Merging items never produces shift/reduce conflicts but may produce reduce/reduce conflicts.
- New conflicts can not be of shift reduce kind:
  - Assume there is a shift reduce conflict in some state of LALR parser with items $[X \to \alpha., a], [Y \to \gamma.a\beta, b]$

Important points

- SLR and LALR parse tables have same number of states.
- Merging items never produces shift/reduce conflicts but may produce reduce/reduce conflicts.
- New conflicts can not be of shift reduce kind:
  - Assume there is a shift reduce conflict in some state of LALR parser with items $[X \to \alpha., a], [Y \to \gamma.a\beta, b]$
  - Then there must have been a state in the LR parser with the same core

Important points

- SLR and LALR parse tables have same number of states.
- Merging items never produces shift/reduce conflicts but may produce reduce/reduce conflicts.
- New conflicts can not be of shift reduce kind:
  - Assume there is a shift reduce conflict in some state of LALR parser with items $[X \to \alpha., a], [Y \to \gamma.a\beta, b]$
  - Then there must have been a state in the LR parser with the same core
  - Contradiction; because LR parser did not have conflicts

Important points

- SLR and LALR parse tables have same number of states.
- Merging items never produces shift/reduce conflicts but may produce reduce/reduce conflicts.
- New conflicts can not be of shift reduce kind:
  - Assume there is a shift reduce conflict in some state of LALR parser with items $[X \to \alpha., a], [Y \to \gamma.a\beta, b]$
  - Then there must have been a state in the LR parser with the same core
  - Contradiction; because LR parser did not have conflicts
- LALR parser can have new reduce-reduce conflicts

Important points

- SLR and LALR parse tables have same number of states.
- Merging items never produces shift/reduce conflicts but may produce reduce/reduce conflicts.
- New conflicts can not be of shift reduce kind:
  - Assume there is a shift reduce conflict in some state of LALR parser with items $[X \to \alpha., a], [Y \to \gamma.a\beta, b]$
  - Then there must have been a state in the LR parser with the same core
  - Contradiction; because LR parser did not have conflicts
- LALR parser can have new reduce-reduce conflicts
  - Assume states $[X \to \alpha., a], [Y \to \alpha., b] and [X \to \alpha., b], [Y \to \alpha., a]$

Important points

- SLR and LALR parse tables have same number of states.
- Merging items never produces shift/reduce conflicts but may produce reduce/reduce conflicts.
- New conflicts can not be of shift reduce kind:
  - Assume there is a shift reduce conflict in some state of LALR parser with items $[X \to \alpha., a], [Y \to \gamma.a\beta, b]$
  - Then there must have been a state in the LR parser with the same core
  - Contradiction; because LR parser did not have conflicts
- LALR parser can have new reduce-reduce conflicts
  - Assume states $[X \to \alpha., a], [Y \to \alpha., b] and [X \to \alpha., b], [Y \to \alpha., a]$
  - Merging the two states produces $[X \to \alpha., a/b], [Y \to \alpha., a/b]$

Important points

- Practical LALR parsers are not built by first making canonical LR parse tables

Important points

- Practical LALR parsers are not built by first making canonical LR parse tables
- There are direct, complicated but efficient algorithms to develop LALR parsers

Important points

- Practical LALR parsers are not built by first making canonical LR parse tables
- There are direct, complicated but efficient algorithms to develop LALR parsers
- Relative power of various classes

Important points

- Practical LALR parsers are not built by first making canonical LR parse tables
- There are direct, complicated but efficient algorithms to develop LALR parsers
- Relative power of various classes
  - SLR(1) $\leq$ LALR(1) $\leq$ LR(1)

Important points

- Practical LALR parsers are not built by first making canonical LR parse tables
- There are direct, complicated but efficient algorithms to develop LALR parsers
- Relative power of various classes
  - SLR(1) $\leq$ LALR(1) $\leq$ LR(1)
  - LALR(k) $\leq$ LR(k)

Important points

- Practical LALR parsers are not built by first making canonical LR parse tables
- There are direct, complicated but efficient algorithms to develop LALR parsers
- Relative power of various classes
  - SLR(1) $\leq$ LALR(1) $\leq$ LR(1)
  - LALR(k) $\leq$ LR(k)
  - LL(k) $\leq$ LR(k)

Error Recovery

- An error is detected when an entry in the action table is found to be empty.

Error Recovery

- An error is detected when an entry in the action table is found to be empty.
- Panic mode error recovery can be implemented as follows:

Error Recovery

- An error is detected when an entry in the action table is found to be empty.
- Panic mode error recovery can be implemented as follows:
  - scan down the stack until a state S with a goto on a particular nonterminal A is found.

Error Recovery

- An error is detected when an entry in the action table is found to be empty.
- Panic mode error recovery can be implemented as follows:
  - scan down the stack until a state S with a goto on a particular nonterminal A is found.
  - Discard zero or more input symbols until a symbol a is found that can legitimately *follow*(A).

Error Recovery

- An error is detected when an entry in the action table is found to be empty.
- Panic mode error recovery can be implemented as follows:
  - scan down the stack until a state S with a goto on a particular nonterminal A is found.
  - Discard zero or more input symbols until a symbol a is found that can legitimately $follow(A)$.
  - stack the state $goto[S, A]$ and resume parsing.

Error Recovery

- An error is detected when an entry in the action table is found to be empty.
- Panic mode error recovery can be implemented as follows:
  - scan down the stack until a state S with a goto on a particular nonterminal A is found.
  - Discard zero or more input symbols until a symbol a is found that can legitimately $follow(A)$.
  - stack the state $goto[S, A]$ and resume parsing.
- Choice of A: Normally these are non terminals representing major program pieces such as an expression, statement or a block. For example if A is the nonterminal stmt, a might be semicolon or end

Parser Generator

- Some common LR parser generators
  - YACC: Yet Another Compiler Compiler
  - Bison: GNU Software
- Yacc/Bison source program specification

declaration
%%
translation rules
%%
supporting C routines

Yacc and Lex schema

